22. Ad Click Event Aggregation

With the rise of Facebook, YouTube, TikTok, and the online media economy, digital advertising is taking an ever bigger share of the total advertising spending. Ad click event aggregation is a critical piece in the ad tech industry. In this chapter, we explore how to design an ad click event aggregation system on Facebook or Google scale.

Ad click event aggregation is a critical piece in the ad tech industry. It is critical because the system has to design an ad click event aggregation system on Facebook or Google scale. Ad click event aggregation is critical because it enables advertisers to make data-driven decisions. For example, they can decide how much to bid for a keyword based on the real-time and aggregate number of click events. Ad click event aggregation is also critical because it enables advertisers to better understand the traffic. Over time, the benefit of online advertising is the measurability, as quantified by real time data.

Digital advertising is a multi-billion dollar industry. In the United States, 67% to 69% of digital advertising inventory is bought and sold. Figure 1 shows how the online ad ecosystem works.

Figure 1 DPA process

The speed of the DPA process is important as it usually occurs in real-time or near-real-time.

Data accuracy is also very important. Ad click event aggregation plays a critical role in measuring the effectiveness of online advertising, which essentially impacts how much money advertisers pay. Based on the click aggregation results, campaign managers can control the budget and bid strategy for their campaigns. For a large ad click event aggregation system, Facebook or Google, billions of ad events are created daily, and billions of aggregated results are created. For example, if Facebook serves 2 billion ad impressions per day, and 1% of these impressions are clicked, then there will be 20 million ad click events. Ad click event aggregation is a challenging problem because it requires processing a large volume of data in real-time, and the data is susceptible to many issues, such as aggregation latency, data delivery, and data integrity.

Step 1 - Understand the Problem and Establish Design Scope

The following are some questions that we can ask the interviewer and various ideas the candidate.

Candidate: What is the format of the input data?

Interviewer: Let's assume the input data is in the JSON format. Each event has a timestamp and a field to denote the ad. That is, the ad. There are the following attributes: ad_id, click_timestamp, user_id, ip, and country.

Candidate: What is the data volume?

Interviewer: 2 billion ad clicks per day and 2 million ads in total. The number of ad click events grows 30% year-over-year.

Candidate: What is the expected query latency for the aggregation?

Interviewer: The aggregation results are used for billing and business reporting. Some of the requirements are:

- Return the number of ad click events for a particular ad in the last 5 minutes.
- Return the top 100 most clicked ads in the past 1 minute. Aggregation window should be configurable. Aggregation occurs every minute.
- Support data filtering by ad, user, ip, or country for the above two aggregations.

Candidate: Do we need to worry about edge cases? I can think of the following:

- There might be events that arrive late at the aggregation service.
- There might be duplicate events.
- Different parts of the system might be down at any time, so we need to consider system recovery.

Interviewer: That is a good list. Yes, take these into consideration.

Candidate: What is the latency requirement?

Interviewer: A few minutes is acceptable for most cases. However, for billing and business reports, the aggregation results are very different from the ad click event data, the latency requirement is stricter. For example, billing should be within a few days. Ad click event aggregation is expensive in terms of storage and computing.

With the information gathered above, we have both functional and non-functional requirements.

Functional requirements

- Aggregate the number of clicks of an ad in the last minute.
- Return the top 100 most clicked ads in the last minute.
- Support aggregation filtering by different attributes.
- Support streaming to a downstream or Google cloud. Use the back of envelope estimation section below for detailed system scale requirements.

Non-functional requirements

- Correctness of the aggregation result is important as the data is used for RTB and ad billing.
- Properly handle delayed or late events.
- Robustness. The system should be resilient to partial failures.
- Latency requirements. End-to-end latency should be a few minutes, at most.

Back-of-the-envelope estimation

Let's assume the traffic volume is similar to Google ad click events and the system overhead is manageable.

- Assume an average each user clicks 1 ad per day. That's 1 billion ad click events per day.
- Ad click QPS = 10^9 events / 10^5 seconds = 1 ads = 10,000.
- Assume the peak QPS is five times the average QPS. Peak QPS = 10,000 * 5 = 50,000 QPS.
- Assume a single ad click event occupies 0.1 KB storage. Ad click event daily storage requirement = 1 billion * 0.1 KB. The monthly storage requirement is about 2 TB.

Step 2 - Propose High-Level Design and Get Buy-In

In this section, we discuss query API design, data model, and high-level design.

Query API design

The purpose of the API design is to have an agreement between the client and the server. In a consumer app, a client is usually the end user who uses the product. In our case however, a client is the dashboard user (data scientist, product manager, advertiser, etc.) who requests the ad click event aggregation results.

Let's review the functional requirements to see what needs to be exposed to the client.

- Aggregate the number of clicks of ad id in the last 60 minutes.
- Return the top 100 most clicked ad in the last 60 minutes.
- Support aggregation filtering by different attributes.

We only need two RPCs for our system. We use cursor because filtering (the last requirement) can be supported by adding query parameters to the request.

API 1: Aggregate the number of clicks of ad_id in the last 60 minutes.

API	Detail
GET /v1/ads/{ad_id}/aggregated_count	Return aggregated event count for a given ad_id

Table 1 API for aggregating the number of clicks

Request parameters are:

Field	Description	Type
from	Start time (default is now minus 1 minute)	long
to	End minutes (default is now)	long
filter	An identifier for different filtering strategies. For example, filter id1 means apply filter strategy id1	long

Table 2 Request parameters for /v1/ads/{ad_id}/aggregated_count

Response:

Field	Description	Type
ad_id	The identifier of the ad	string
count	The aggregated count between the start and end minutes	long

Table 3 Response for /v1/ads/{ad_id}/aggregated_count

API 2: Return top 100 most clicked ads in the last 60 minutes

API	Detail
GET /v1/ads/popular_ads?count=x&min=y	Return top N most clicked ads in the last M minutes

Table 4 API for /v1/ads/popular_ads

Request parameters are:

Field	Description	Type
count	Top N most clicked ads	integer
minutes	The aggregation window size (M in minutes)	long
filter	An identifier for different filtering strategies	long

Table 5 Request parameters for /v1/ads/popular_ads

Response:

Field	Description	Type
ad_ids	A list of the most clicked ads	array

Table 6 Response for /v1/ads/popular_ads

Data model

There are two types of data in the system: raw data and aggregated data.

Raw data

Below shows what the raw data looks like by log files:

```
[ad1:clicks] ad001, 2021-01-01 00:00:01, user1, 207.148.22.22, ios
```

Table 7 lists what the data looks like in a structured format. This is a common format in different application services.

ad_id	click_timestamp	user	ip	country
ad001	2021-01-01 00:00:01	user1	207.148.22.22	cm
ad001	2021-01-01 00:00:02	user1	207.148.22.23	cm
ad002	2021-01-01 00:00:02	user2	200.162.192.1	bh

Table 7 Raw data

Aggregated data

Assume the ad click events are aggregated every minute. Table 8 shows the aggregated result.

ad_id	click_minute	count
ad001	202101010000	5
ad002	202101010000	7

Table 8 Aggregated data

Table 9 shows the aggregated data with filter.

filter_id	region	ad_id	cnt_id
0012	cn	...	...
0013	cn	...	...
...	...	...	...

Table 9 Filter data

To support the query in where the top K most clicked ads in the last M minutes, the following structure is used:

name_interval_idx	
window_end	bigint
top_clicked_ads	varchar

Table 10 Aggregate window size (M in minutes)

update_time_minute	timestamp	Last updated timestamp for a minute processing
most_clicked_ads	array	List of ad ids in JSON format

Table 11 Support top N most clicked ads in the last M minutes

Comparison

Comparison between storing raw data and aggregated data is shown below:

	Raw data	Aggregated data
Pros	• Full data set • Support data filtering and recalculation	• Smaller data set • Fast query
Cons	• Huge data storage • Slow query	• Data loss. This is derived data. For example, 1D window might be aggregated for 1 hour.

Table 12 Raw data vs aggregated data

Should we store raw data or aggregated data? We recommend/assume to store both, and it takes a look at why.

- It's a good idea to keep the raw data. If something goes wrong, we can rerun the job to fix the data/bug. If the aggregated data is the only data we have, it is difficult to fix the data if it is not correct because some of the original data is lost.
- Aggregated data should be derived by using the raw data in a large data set. The large data volume ensures we can make data-driven decisions sufficiently. To mitigate the problem, we can store quantiles or aggregated clicks.
- Raw data serves as backup data. We don't want to send a data reprocessing request to upstream if we want to recompute a new data set. Instead, we can reprocess the data in the data lake.
- Aggregated data serves as ad-hoc data to meet query performance requirements.

Choose the right database

When it comes to choosing the right database, we need to evaluate the following:

- What does the data look like? Is the data relational? Is it a document or a blob?
- What is the read/write ratio? Do we need to optimize it?
- Is transaction support needed?
- Do the queries rely on many other analytical processing (OLAP) functions (GS like SUM, COUNT)?

Let's assume the raw data is stored in Kafka and we want to store the raw data for different applications. It is useful for data scientists or machine learning engineers to study user response predictors, behavioral targeting, relevance feedback, etc. [1].

We don't expect to use relational data or document data for this system. The data model we use is key-value data, and our system is read-heavy. The data volume is huge, and both read and write can reach peak quickly because ad click events happen in real-time.

Relational datasets can do the job, but scaling the write can be challenging. Kafka, alias Aerospike like Cassandra and InfluxDB are more suitable because they are optimized for time-series and aggregation.

Another option is to store the data in Amazon S3 using one of the storage classes like S3 Glacier, Flexible, or S-IAR-OZ. We could put a cap on the ad id and the day, 50GB, and the other process responsible for writing the raw data could handle the file rotation. It is ideal for the data lake, but it is not ideal for the query because it takes a long time to get the data into a query engine.

For aggregated data, it is time series in nature and the workflow is both read and write heavy. This is because, for each ad, we need to update the aggregated count every minute based on the ad click event data. The workload is quite OLAP and write-heavy. We have decided on a time-series database (TSDB) as our choice. The database shifts quite well from the message queue, transforms the data into the datastore format, and writes to the TSDB.

Figure 2 Kafka and time-series database

What's stored in the first message queue? It contains the raw ad click event data as shown in Table 13.

ad_id	click_timestamp	user_id	ip	country

Table 13 Data in the first message queue

What is stored in the second message queue? The second message queue contains the types of data:

- No ad count aggregation at per minute granularity.

ad_id	click_minute	count

Table 14 Data in the second message queue

2. Top N most clicked ads aggregated at per minute granularity.

update_time_minute	most_clicked_ads

Table 15 Data in the second message queue

You might be wondering why we don't write the aggregated results to the database directly. The short answer is that we need the second message queue for fault tolerance, data consistency, and to support exactly-once semantics.

Figure 3 Kafka and time-series database

Next, let's dig into the details of the aggregation service.

Aggregation service

The MapReduce framework is a good option to aggregate ad click events. The directed acyclic graph (DAG) is a good model for it [2]. The design of our DAG is broad inspired by Google ad click event aggregation paper (F1), the MapReduce/Spark/Flink clones, as shown in Figure 4.

Figure 4 Aggregation service

Each node is responsible for one single task and it sends the processing result to its downstream nodes.

Node 1

It takes node status data from a data source, and then filters and transforms the data. For example, a step node sends ad1-ad2 to *,1 / 2 for node 1 and the other ads go to node 2, as shown in Figure 5.

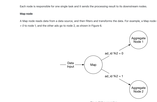


Figure 5 Map operation

You might be wondering why we need the Map node. An alternative option is to set up traffic partitions or tags and let the aggregate node consume the data of its own partition. For example, node 1 only produces ad1 and ad2 data and node 2 only produces ad3 and ad4 data by the filter node. Another reason is that we may not have control over how data is produced and therefore events with the same ad_id might end in different traffic partitions.

Aggregate node

An Aggregate node counts ad click events by ad_id in memory every minute. In the MapReduce paradigm, the Aggregate node is part of the reducer node. The final aggregate result is written to a message queue.

Reduce node

The Reduce node is responsible for aggregating intermediate aggregate results in the final result. For example, as shown in Figure 6, there are three aggregation nodes and each contains the top 3 most clicked ad in the last minute. The Reduce node receives the most clicked ad result from all three nodes.

Figure 7 Reduce node

The DAG model represents the well-known MapReduce paradigm. It is designed to strike big data and use parallel distributed computing to turn big data into little. A regular small data.

In the DAG model, intermediate data can be stored in memory and different nodes communicate with each other through either TCP (node-running in different processes or shared memory) (node-running in different threads).

Main use cases

Now that we understand the MapReduce works at the high level, let's take a look at how it can be utilized to support the main use cases.

- Aggregate the number of clicks of ad(s) in the last .M mins.
- Return top N most clicked ad(s) in the last .M minutes.
- Data Filtering.

Use case 1: aggregate the number of clicks

As shown in Figure 8, filter and aggregate are performed by ad_id and ad_id%N. D. Map nodes and are their aggregation nodes.

Figure 8 Aggregate the number of clicks

Use case 2: return top N most clicked ads

Figure 9 shows a smarter design of getting the top 3 most clicked ads, which can be extended to be N. Hard events are mapped using ad_id and aggregate node maintains a heap data structure to track the top 3 ads within the node efficiently. In the last step, the Reduce node retrieves 9 ads (3*3=9) from each aggregate node to the top 3 most clicked ads every minute.

Figure 9 Aggregate the number of clicks

Use case 3: data filtering

To support data filtering for faster recovery the aggregated data (count for ad001 within the last 1 minute), we can use data filtering criteria and aggregate based on them. For example, the aggregation results look like the ad001 and ad002.

ad_id	click_minute	country	count
ad001	202101010000	cm	100
ad001	202101010000	GPB	200
ad002	202101010000	uhaul	10
ad001	202101010001	cm	120
ad001	202101010001	GPB	210
ad001	202101010001	uhaul	11

Table 16 Aggregation results (filter by country)

This technique is called the star schema [3], which is widely used in data warehouse. The following fields are added dimensions. The aggregation results support the following operations:

- The current aggregation window can be broad to create more dimensions in the star schema. For additional component is needed.
- Increasing data based on filtering criteria is first because the result is pre-calculated.

A dashboard with aggregation results that it creates many more buckets and records, especially when we have a lot of filtering criteria.

Step 3 - Design Deep Dive

In this section, we will dive deeper into the following topics:

- Streaming vs batching
- Time-ad aggregation window
- Delivery guarantees
- Scale the system
- Data reconciling and corrections
- Data monitoring
- Fault tolerance

Streaming vs batching

The high-level architecture we proposed in Figure 3 is a type of stream processing system. Table 17 shows the comparison of three types of systems [5].

	Services (Online systems)	Batch system (offline system)	Streaming system (near-real time system)
Responsiveness	Response to the client quickly	No response to the client needed	No response to the client needed
Input	User requests	Bounded input with finite size, A large amount of data	Input has no boundary (infinite streams)
Output	Responses to clients	Materialized views, aggregation metrics, etc.	Materialized views, aggregation metrics, etc.
Performance measurement	Availability, latency	Throughput	Throughput, latency
Example	Online shopping	MapReduce	Flink [4]

Table 17 Comparison of three types of systems

In our design, both stream processing and batch processing are used. We utilized stream processing to process data as it arrive and operations aggregation results in near-real time with a few minutes of latency. Batch processing is the historical data backup.

For a system that considers two processing paths (batch and streaming) simultaneously, this architecture is called lambda [5]. It's a popular design that gives us a robust system. The other architecture we considered is called kappa. In the kappa architecture, only one processing path is considered. Lambda [5], which combines fast and slow processing, is a more popular option. The key idea is to handle data that is sensitive to latency in a stream processing manner, and to process other data when it is ready in a batch processing manner.

For a system that considers two processing paths (batch and streaming) simultaneously, this architecture is called lambda [5]. It's a popular design that gives us a robust system. The other architecture we considered is called kappa. In the kappa architecture, only one processing path is considered. Lambda [5], which combines fast and slow processing, is a more popular option. The key idea is to handle data that is sensitive to latency in a stream processing manner, and to process other data when it is ready in a batch processing manner.

Lambda Architecture

Figure 12: Lambda and Kappa Architecture

Our high level design uses Kappa architecture, where the reprocessing of historical data also goes through the real-time aggregation service. See the "Data recalculation" section below for details.

Data recalculation

Sometimes we have to recalculate the aggregated data, also called historical data replay. For example, if we discover a mapp bug in the aggregation service, we want to recalculate the aggregated data from the date starting at the point where the bug was introduced. Recalculation is CPU intensive and may affect other services.

1. The recalculation service retrieves data from raw data storage. This is a batch job.
2. Retrieved data is sent to a dedicated aggregation service so that the real-time aggregation is not impacted by historical data replay.
3. Aggregated results are sent to the second message queue. Then updated in the aggregation database.

Figure 13: Recalculation service

The recalculation process routes the aggregation service not uses a different data source (the raw data).

Time

We need a timestamp to perform aggregation. The timestamp can be generated in two different places.

- Event time, the time when an event happens.
- Processing time, the time a system notice of the event when the event arrives the first server.

Due to network delays and asynchronous environments (data go through a message queue), the gap between event time and processing time can be large. As shown in Figure 15, event 2 arrives at the aggregation service very late.

Figure 15: Late events

If event time is used for aggregation, we have to deal with delayed events. If processing time is used for aggregation the aggregation result may not be accurate. There is no perfect solution, so we need to consider the trade-offs.

	Pros	Cons
Event time	Aggregation results are time-accurate because aggregation calculations are done on a timed basis.	It depends on the timeliness generated on the client side. Clients might be in a different time zone or even set to a timed.
Processing time	Server timestamp is more reliable.	The timestamp is not accurate if an event reaches the system at a much later time.

Table 8A: Event time vs processing time

Since data is never lost in Kappa, we decided using event time for aggregation. You can also use both processing time and event time in this case if a technique called "watermark" is commonly utilized to handle slightly delayed events.

In Figure 16, we set a watermark for 15 minutes after processing the first event. The watermark acts as a buffer. Any events that arrive within the watermark period will be aggregated. In this example, window 1 includes event 1 and event 2. Window 2 contains only event 3 because it arrives outside the watermark period even though it is less than 15 minutes late.

Figure 16 Watermark

Notice that the watermark technique does not handle events that have long delays. We can argue that it is not worth the return on investment (ROI) to have a comprehensive design for late aggregation. Most always correct data is important for ads billing and other data reconciliation uses. Real-time dashboard or operational metrics do not need to be 100% accurate. Slightly delayed or even lost events are acceptable in these cases.

Aggregation window

According to the "Designing data-intensive applications" book by Martin Kleppmann, there are four types of window functions: tumbling (also called fixed) window, hopping window, sliding window, and session window. We will discuss the tumbling window and hopping window because they are most applicable.

In a tumbling window (also called fixed window) as shown in Figure 17, time is partitioned into fixed length, non-overlapping chunks. The tumbling window is a good fit for aggregating event metrics by time-to-live (TTL).

Figure 17: Tumbling window

In the sliding window (highlighted in Figure 18), events are grouped within a window that slides across the data stream, according to a specified interval. An event can belong to more than one window. For example, if delta is set to 1 minute, we can slide a window to get the top 10 most clicked ads during the last 10 minutes.

Figure 18 Sliding window

Delivery guarantees

Since the aggregation result is utilized for billing, data accuracy and completeness are very important. The system needs to be able to answer questions such as:

- How is exact-processing duplicate events?
- How to prevent data loss or duplication?

Message queues such as Kafka usually provide three delivery semantics: at most once, at least once, and exactly once.

Which delivery semantic do we choose?

In most circumstances, at least once processing is good enough if a small percentage of duplicates are acceptable.

Exactly once is ideal for the ad click aggregation service because billing is based on the ad click counts. Even a slight discrepancy of critical metrics, like billing, can accumulate to a large number when it is processed on a massive scale. Ad click aggregation is a perfect fit for exactly-once delivery because it deals with financial data.

Data deduplication

Ad click events from the ad click event ingestion service. Duplicated data can come from a wide range of sources and in this service, we discuss two common scenarios.

- Client side. For example, a client might send the same ad click event multiple times. Duplicated events with nonfunctional fields are hard to handle on the aggregation components. It is not unusual to place code in the reference material SDK.
- Server outage. If an aggregation service node goes down in the middle of aggregation and the upstream service hasn't yet received the acknowledgment, another aggregation service node will process the same ad click event.

Figure 19 shows that aggregation service nodes frequently update their current offset in Kafka. The Aggregator changes the status of data corresponding to placing the offset in upstream Kafka.

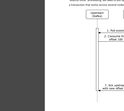


Figure 19 Kafka offset

If step 5 fails, perhaps due to Aggregator outage, events from 100 to 110 are already sent to the downstream, but the new offset 110 is not persisted in upstream Kafka. In this case, a new Aggregator instance will pick up the offset from upstream Kafka and process all the already processed, causing duplicate data.

The most straightforward solution (Figure 19) is to use external file storage, such as HDFS or S3, to record the offset. However, this solution has issues as well.

Figure 20 Record the offset

In step 3, the aggregator will process events from offset 100 to 110, and it has set offset stored in external storage to 100 if the offset doesn't in the storage in TTL, the aggregator ignores events before offset 110.

But the danger has a major problem that the offset is saved in HDFS / S3 before the aggregation result is sent downstream. If step 6 fails due to aggregator outage, events from 100 to 110 will never be processed by a newly brought up aggregator node, since the offset persisted in external storage is 110.

To avoid data loss, we need to save the offset once we get an acknowledgment back from downstream. The updated design is shown in Figure 21.

Figure 21 Save offset after receiving ack

In this design, if the Aggregator is down before step 5 is executed, events from 100 to 110 will be sent downstream again. To achieve exactly-once delivery, we need to find a way to persist the offset and the aggregation result in the same transaction. Unfortunately, this is a cross-system transaction and is hard to implement. As a workaround, the offset transaction is stored in Kafka.

Figure 22 Kafka transaction

As you can see, it's not easy to deliver Kappa to large scale systems. Time to achieve exactly-once processing is an aforementioned topic. If you are interested in this details, please refer to reference material [9].

Scale the system

From the back of the envelope estimation, we know the business grows 30% per year, which doubles traffic every 3 years. How do we handle the growth? Let's take a look.

Our system consists of three independent components: message queue, aggregation service, and database. Since these components are decoupled, we can scale each one independently.

Scale the message queue

We have already discussed how to scale the message queue extensively in the "Distributed Message Queue" chapter, so we'll only briefly touch on a few points.

Producers. We don't limit the number of producer instances, so the scalability of producers can be easily achieved.

Consumers. Inside a consumer group, the rebalancing mechanism helps to scale the consumers by adding or removing nodes. As shown in Figure 23, the rebalancing mechanism distributes partitions among the consumers in the group.

Topic	
Partition 1	Consumer 1
Partition 2	Consumer 2
Partition 3	Consumer 3
Partition 4	Consumer 4

Rebalance

Topic	
Partition 1	Consumer 1
Partition 2	Consumer 1
Partition 3	Consumer 2
Partition 4	Consumer 2

Figure 23 Kafka consumers

When there are hundreds of Kafka consumers in the cluster, consumer rebalance can be quite slow and could take a few minutes even. Therefore, if more consumers need to be added, try to do it during off-peak hours to minimize the impact.

Brokers

- Hashing key

Using an ad id as hashing key helps the system to store events from the same ad in the same Kafka partition. In this case, an aggregation service can subscribe to all events of the same ad id from one single partition.

- The number of partitions

If the number of partitions changes, events of the same ad id might be mapped to a different partition. Therefore, the system needs to allocate enough partitions to avoid dynamically increasing the number of partitions in production.

- Topic physical sharding

One single topic is usually not enough. We can shift the data by geography (topic_north_america, topic_europe, topic_apac, etc.) or by business type (topic_text_ads, topic_image_ads, etc.).

Hashing data to different topics can help increase the system throughput. With these consumers for a single topic, the time to rebalance consumer groups is tolerable.

Cons: It introduces extra complexity to clients and increases maintenance costs.

Scale the aggregation service

In the high level design, we talked about the aggregation service using a map/reduce operation. Figure 24 shows how things are wired together.

Figure 24 Map/reduce

If you are interested in the details, please refer to reference material [10]. Aggregation service is horizontally scalable by adding or removing nodes. Here are interesting questions: how do we increase throughput of the aggregation service? Read on for two options.

Option 1: Allocate events with different ad_id to different threads, as shown in Figure 25.

Figure 25 Consistent hashing

Data is evenly distributed to every node with a proper replication factor. Each node spawns its own part of the ring based on hashed value and also spawns multiple threads (vnodes).

If we add a new node to the cluster, it automatically rebalances the virtual nodes among all nodes. No manual rebalancing is required. See Consistent hashing section in the "Ad click event aggregation" chapter for more details [11].

Fault tolerance

Aggregating around one minute result more data than the others is called a hotspot. This occurs because major companies have advertising budgets in the millions and spend dollars per minute. Ad click events from these ads are partitioned by ad id, some aggregation service nodes might receive many more ad click events than others, potentially causing server overload.

The problem can be mitigated by allocating more aggregation threads to process popular ads, until a new node is ready to join the cluster. In Figure 26, the event aggregation node can handle only 100 events.

- Ad id 1 is a popular ad. The aggregation node dispatch the capacity of a node (see handle). It applies for extra resources through the cluster manager.
- The resource manager allocates new resources (for example, add two more aggregation nodes) so the original aggregation node can handle the workload.
- The original aggregation node will process only the popular ad id and aggregation node handles 100 events.
- The hash ring in consistent hashing is the original aggregation node.

Resource Manager
Ad Events
Hash resource allocation table
Aggregation Node
Aggregation Node
Aggregation Node
Hash ring
Resource Pool
Aggregation Node
Aggregation Node
Aggregation Node

Figure 29 Alternative design

For more detail on this, please refer to reference material [26].

Step 4 - Wrap Up

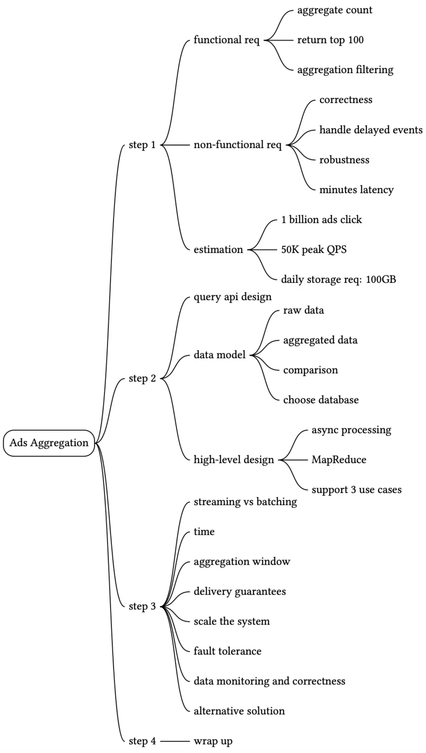
In this chapter, we went through the process of designing an ad click event aggregation system at the scale of Facebook or Google. We covered:

- Data model and API design.
- Use MapReduce paradigm to aggregate ad click events.
- Scale the message queue, aggregation service, and database.
- Mitigate hotspot issue.
- Monitor the system continuously.
- Use reconciliation to ensure correctness.
- Fault tolerance.

The ad click event aggregation system is a typical big data processing system. It will be easier to understand and design if you have prior knowledge or experience with industry-standard solutions such as Apache Kafka, Apache Flink, or Apache Spark.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Chapter Summary



Reference Materials

[1] Clickthrough rate (CTR): Definition: https://support.google.com/google-ads/answer/2615875?hl=en

[2] Conversion rate: Definition: https://support.google.com/google-ads/answer/2684489?hl=en

[3] OLAP functions: https://docs.oracle.com/database/121/OLAXS/olap_functions.htm#OLAXS169

[4] Display Advertising with Real-Time Bidding (RTB) and Behavioural Targeting: https://arxiv.org/pdf/1610.03013.pdf

[5] LanguageManual ORC: https://cwiki.apache.org/confluence/display/hive/languagemanual+orc

[6] Parquet: https://databricks.com/glossary/what-is-parquet

[7] What is avro: https://www.ibm.com/topics/avro

[8] Big Data: https://www.datakwery.com/techniques/big-data/

[9] DAG model https://en.wikipedia.org/wiki/Directed_acyclic_graph

[10] Java stream: https://docs.oracle.com/javase/8/docs/api/java/util/stream/Stream.html

[11] Understand star schema and the importance for Power BI: https://docs.microsoft.com/en-us/power-bi/guidance/star-schema

[12] Martin Kleppmann, "Designing Data-Intensive Applications", 2017

[13] Apache Flink: https://flink.apache.org/

[14] Lambda architecture: https://databricks.com/glossary/lambda-architecture

[15] Kappa architecture: https://hazelcast.com/glossary/kappa-architecture/

[16] Martin Kleppmann, "Stream Processing, Designing Data-Intensive Applications", 2017

[17] End-to-end Exactly-once Aggregation Over Ad Streams: https://www.youtube.com/watch?v=hzxytnPcAUM

[18] Ad traffic quality: https://www.google.com/ads/adtrafficquality

[19] An Overview of End-to-End Exactly-Once Processing in Apache Flink: https://flink.apache.org/features/2018/03/01/end-to-end-exactly-once-apache-flink.html

[20] Understanding MapReduce in Hadoop: https://www.section.io/engineering-education/understanding-map-reduce-in-hadoop/

[21] Flink on Apache Yarn https://ci.apache.org/projects/flink/flink-docs-release-1.13/docs/deployment/resource-providers/yarn/

[22] How data is distributed across a cluster (using virtual nodes): https://docs.datastax.com/en/cassandra-oss/3.0/cassandra/architecture/archDataDistributeDistribute.html

[23] Flink performance tuning: https://nightlies.apache.org/flink/flink-docs-master/docs/dev/table/tuning/

[24] ClickHouse: https://clickhouse.com/

[25] Druid: https://druid.apache.org/

[26] Real-Time Exactly-Once Ad Event Processing with Apache Flink, Kafka, and Pinot: https://eng.uber.com/real-time-exactly-once-ad-event-processing/

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.